

Programs asked in Companies during Technical Rounds

Sr.	Program
1 (M)	Dominant Number in an Array A dominant number is a number whose frequency of occurrence in an array is strictly greater than the count of unique elements present in that array. You are given an integer array. Your task is to identify and return all dominant numbers from the array. Input: arr = [1, 2, 3, 1, 1, 4, 5, 1, 1, 1] Output: [1] Input: arr = {1, 1, 1, 2, 2, 2, 3} Output: [1, 2]
2 (E)	Find Maximum, Second Maximum, Minimum, and Second Minimum from an Array This program takes an array of integers as input and identifies: - the Maximum element, - the Second Maximum element, - the Minimum element, and - the Second Minimum element. The array must contain at least four distinct elements to determine all four values correctly. Input: Array = [10, 5, 20, 8, 15] Output: Maximum = 20 Second Maximum = 15 Minimum = 5 Second Minimum = 8 Input: Array = [3, 7, 1, 9, 4] Output: Maximum = 9 Second Maximum = 7 Minimum = 1 Second Minimum = 3
3 (E)	Time Problem Create a Time class to represent time in hours, minutes, and seconds. - Initialize Hr : Min : Sec using a constructor. - Create an AddSeconds(seconds) method to add the given seconds to the initialized time. - Create a SubtractSeconds(seconds) method to subtract the given seconds from the initial time. - Ensure proper time adjustment (60 seconds = 1 minute, 60 minutes = 1 hour). - Print the time in Hr:Min:Sec format. Input: Initial Time = 02 : 30 : 45 Add Seconds = 50 Subtract Seconds = 30 Output: Time after adding seconds = 02 : 31 : 35 Time after subtracting seconds = 02 : 31 : 05 Input: Initial Time = 01 : 15 : 20

	Add Seconds = 100 Subtract Seconds = 40 Output: Time after adding seconds = 01 : 17 : 00 Time after subtracting seconds = 01 : 16 : 20
4 (M)	Maximum Price Selection Problem You are given a list of items along with their prices and categories. The values N and K are also provided: N → Maximum number of items that can be selected K → Maximum number of times the same Category can be selected repeatedly Each item is represented in the format: price:category The program should select items such that: - At most N items are chosen - Each item can be chosen at most K times - The total price is maximum Finally, return the maximum total price.
	Input: N = 4 K = 2 Input Array = ["500:Electronics", "400:Electronics", "300:Electronics", "200:Sports", "150:Beauty", "100:Beauty"] Output: Maximum Price = 1250 Input: N = 3 K = 1 Input Array = ["600:Electronics", "350:Sports", "250:Beauty", "150:Books"] Output: Maximum Price = 1200
5 (M)	Sentence Similarity Ranking Based on a Secret Word You are given a secret code word and a list of sentences. Your task is to analyse each sentence and calculate how closely it matches the secret word based on the following conditions: Matching Conditions For each word in a sentence, compare it with the secret word: Length must be the same as the secret word Characters must match Positions of matching characters must be the same - Comparison is case-insensitive Each matching character at the correct position increases the similarity score by 1. Overall Task - Calculate the total similarity score for each sentence - Store all similarity scores in an array - Sort the sentences in descending order of similarity - Return the final decoded string by joining sentences according to their similarity rank
	Input: Secret Word = "cat"

Sentences = ["I love my pet",
"A car ran fast",
"He bought a hat",
"The cat sat on the mat"]

Word	Compared with "cat"	Matching Score
The	No match	0
Cat	All letters & positions match	3
Sat	'a' and 't' match in position	2
On	Length mismatch	0
The	No match	0
Mat	'a' and 't' match in position	2

Total Similarity = 7

Similarly, similarity scores for all sentences: [7, 3, 2, 1]

Output:

Final Decoded String = "The cat sat on the mat A car ran fast He bought a hat I love my pet"

6 (H) Find All Harmonious Substrings

Objective

The goal of the program is to **generate a collection** of all possible substrings from a given input string that satisfy the specific "Harmonious" criteria.

The "Harmonious" Criteria

For any individual substring to be included in the final list, the frequency of its characters must adhere to the following rule:

The difference between the maximum character frequency and the minimum character frequency within that substring must be 0 or 1.

Selection Logic:

1. Break the input string into every possible contiguous substring.
2. For each substring, calculate Max_Frequency - Min_Frequency.
3. If the result is ≤ 1 , **add it to the output list.**

Input: String = "ABA"

Output: ["A", "B", "A", "AB", "BA", "ABA"]

Input: String = "AAAB"

Output: ["A", "A", "A", "B", "AA", "AA", "AB", "AAA", "AAB"]

7 (M) Simqual String Problem

Two strings are called **Simqual strings** if they satisfy the following conditions:

1. Both strings must have the **same length**
2. Both strings must contain the **same characters**
3. The **frequency of each character must be equal** in both strings
4. **Order of characters does not matter**

The program should check whether the given two strings are **Simqual** or **Not Simqual**.

Input: String 1 = "listen"

String 2 = "silent"

Output: Strings are Simqual

Input: String 1 = "hello"

String 2 = "world"

	Output: Strings are NOT Simqual
8 (H)	Coin Change Problem Write a program that, given an amount and available coin denominations, calculates the minimum number of coins required to make that amount. Input: Amount = 27 Denominations = {1, 2, 5, 10} Output: Minimum coins needed: 4 (Breakdown: 10 + 10 + 5 + 2) Input: Amount = 23 Denominations = {1, 2, 7} Output: Minimum coins needed = 5 (Breakdown = 7 + 7 + 7 + 1 + 1)
9 (E)	Check for Balanced Brackets Write a program to check whether a given expression containing brackets is balanced or not. Allowed Brackets () { } [] Balancing Rules • Every opening bracket must have a corresponding closing bracket • Brackets must be closed in the correct order • Mixed types of brackets are allowed The program should return whether the expression is Balanced or Not Balanced. Input: Expression = "[()]" Output: Balanced Input: Expression = "{[()]}" Output: Not Balanced
10 (M)	Rotate an N×M Matrix by 90 Degrees Clockwise Write a program to rotate a dynamic matrix of size N×M (not necessarily a square matrix) by 90 degrees clockwise. Rotation Rules • The number of rows and columns may be different. • After rotation, the matrix size becomes M×N. • Elements must be repositioned correctly to represent a 90° clockwise rotation. Input: 1 2 3 4 5 6 7 8 9 Output: 7 4 1 8 5 2 9 6 3 Input: 1 2 3 4 5 6 7 8 Output: 5 1 6 2 7 3 8 4
11 (H)	Form the Largest Number from an Array

	You are given a list of non-negative integers. Arrange them in such an order that they form the largest possible number. Important Rules - Numbers must be arranged, not added - The final result can be very large, so return it as a string - Concatenate numbers in an order that gives the maximum value
	Input: nums = [10, 2] Output: "210" Input: nums = [3, 30, 34, 5, 9] Output: "9534330"
12 (M)	Jump Game – Reach the Last Index You are given an integer array nums where each element represents the maximum jump length from that position. - You start at the first index (index 0). - From any index, you can jump forward up to the number of steps given by the value at that index. - Determine whether it is possible to reach the last index of the array. The program should return: True if the last index can be reached False otherwise
	Input: nums = [2, 3, 1, 1, 4] Explanation - Jump from index 0, Value (2) → index 2, Value(1) - Jump from index 2, Value (1) → index 3, Value(1) - Jump from index 3, Value (1) → index 4, (Last Index) Output: True Input: nums = [3, 2, 1, 0, 4] Explanation - You reach index 3, but the value is 0 - No further jump is possible Output: False
13 (H)	Best Time to Buy and Sell Stock You are given an integer array prices, where prices[i] represents the price of a stock on the ith day. Rules - You may buy and/or sell the stock on any day. - You can hold at most one share of the stock at any time. - You are allowed to buy and sell on the same day. - You may complete multiple transactions. The task is to calculate and return the maximum profit that can be achieved.
	Input: Prices = [7, 1, 5, 3, 6, 4] Explanation - Buy on day 2 at price 1, sell on day 3 at price 5 → profit = 4 - Buy on day 4 at price 3, sell on day 5 at price 6 → profit = 3 - Total profit = 4 + 3 = 7 Output: 7

Input: Prices = [1, 2, 3, 4, 5]

Explanation

- Buy on day 1 and sell on day 5
- Total profit = (2-1) + (3-2) + (4-3) + (5-4) = 4

Output: 4

**14
(H)**

Maximum Students Taking Exam

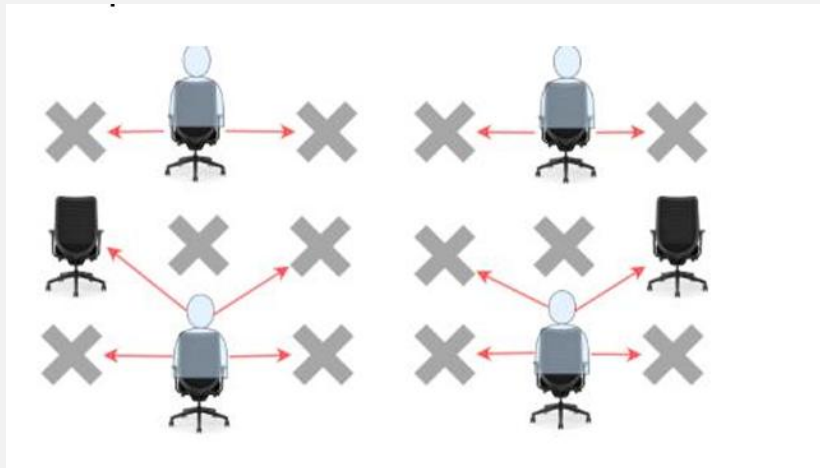
Given an $m \times n$ matrix seats representing a classroom:

- A seat is **broken** $\rightarrow$ '#'
- A seat is **available** $\rightarrow$ '.'

Rules for placing students:

1. Students can **see the answers** of those sitting:
 - Left and right
 - Upper-left and upper-right
2. Students **cannot see** the answers of those directly in front or behind.
3. Students can only be placed on **available seats** ('.').

Return the **maximum number of students** that can take the exam **without cheating**.



Input: seats = [['#', '.', '#', '#', '.', '#'],
['.', '#', '#', '#', '#', '.'],
['#', '.', '#', '#', '.', '#']]

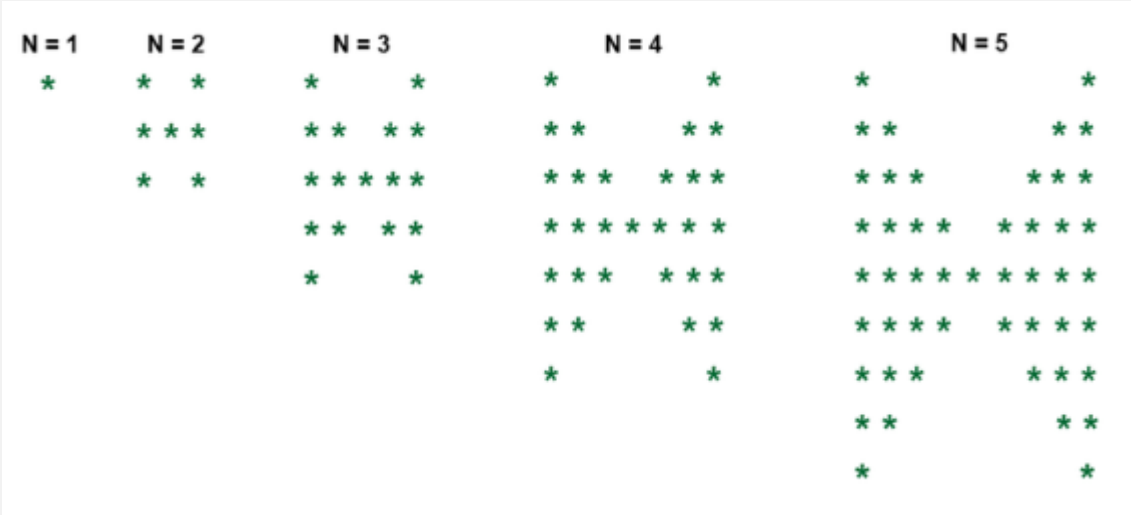
Output: 4

Input: seats = [['.', '#'],
['#', '#'],
['#', '.'],
['#', '#'],
['.', '#']]]

Output: 3

Input: seats = [['#', '.', '.', '.', '#'],
['.', '#', '.', '#', '.'],
['.', '.', '#', '.', '.'],
['.', '#', '.', '#', '.'],
['#', '.', '.', '.', '#']]]

Output: 10

15 (M)	Find the Repeated Number You are given an integer array nums containing n + 1 elements. Each element in the array is in the range [1, n]. - There is exactly one number that is repeated in the array. - The repeated number may appear more than once. - Your task is to identify and return the repeated number. The solution should run in O(n) time complexity.
	Input: nums = [1, 3, 4, 2, 2] Output: 2 Input: nums = [3, 1, 3, 4, 2] Output: 3
16 (M)	Butterfly Pattern The butterfly pattern is a symmetric star (*) pattern that looks like the shape of a butterfly. It is usually printed using two mirrored triangles.
	 The image shows the butterfly pattern for N=1 to N=5. Each pattern consists of two mirrored triangles of stars. For N=1, it's a single star. For N=2, it's two stars. For N=3, it's three stars. For N=4, it's four stars. For N=5, it's five stars. The patterns are arranged in a row, with N=1 on the left and N=5 on the right.
17 (E)	Check Perfect Number A perfect number is a positive integer that is equal to the sum of its positive divisors, excluding the number itself. Given an integer num, determine whether it is a perfect number. - Return true if num is a perfect number - Return false otherwise The solution should run in O(Vn) time complexity and O(1) Space Complexity
	Input: num = 28 Output: true Input: num = 7 Output: false
18 (M)	Square Root of a Number (Sqrt(x)) Given a non-negative integer x, write a program to compute and return the square root of x, rounded down to the nearest integer. Constraints Do not use any built-in square root or exponent functions/operators.

	The result must be the floor value of $\sqrt{x}$. The solution should run in $O(\log x)$ time complexity and $O(1)$ Space Complexity
	Input: $x = 4$ Output: 2 Input: $x = 8$ Output: 2 Explanation: $\sqrt{8} = 2.828\dots$, round down to 2
19 (E)	Check Prime Number A prime number is a natural number greater than 1 that has exactly two positive divisors: 1 and the number itself. Given an integer x, write a program to determine whether the number is prime or not. - Return true if x is a prime number - Return false otherwise The solution should run in $O(n)$ time complexity and $O(1)$ Space Complexity
	Input: $x = 7$ Output: true Input: $x = 10$ Output: false
20 (M)	Factorial Trailing Zeroes Given a non-negative integer n, write a program to calculate the number of trailing zeroes in $n!$ (n factorial). The factorial of a number is defined as: $n! = n \times (n - 1) \times (n - 2) \times \dots \times 2 \times 1$ Trailing zeroes are the continuous zeros at the end of the factorial result. The solution should run in $O(n)$ time complexity and $O(1)$ Space Complexity
	Input: $n = 3$ Output: 0 Explanation: $3! = 6$ Input: $n = 5$ Output: 1 Explanation: $5! = 120$, one trailing zero Input: $n = 0$ Output: 0
21 (M)	Next Greater Element Given an array of integers $arr[]$, find the Next Greater Element (NGE) for each element in the array, in the order of their appearance. The Next Greater Element for an element is defined as: - The first element to the right of it that is greater than the current element. - If no such element exists, return -1 for that position.
	Input: $arr[] = [1, 3, 2, 4]$ Output: $[3, 4, 4, -1]$ Explanation - Next greater of 1 $\rightarrow$ 3 - Next greater of 3 $\rightarrow$ 4

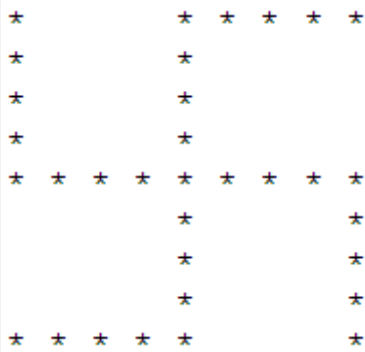
	- Next greater of 2 → 4 - No greater element after 4 → -1 Input: arr[] = [6, 8, 0, 1, 3] Output: [8, -1, 1, 3, -1] Explanation - Next greater of 6 → 8 - No greater element after 8 → -1 - Next greater of 0 → 1 - Next greater of 1 → 3 - No greater element after 3 → -1
22 (E)	Find Nth Factor of a Number Given a positive integer n and an integer k, write a program to find the kth factor of n. - Factors of a number are the positive integers that divide the number exactly. - Factors are considered in ascending order. - If the kth factor does not exist, return -1.
	Input: n = 12, k = 1 Output: 1 Input: n = 12, k = 5 Output: 6 Input: n = 12, k = 3 Output: 3
23 (M)	Arrange Array Elements in Ascending and Descending Order Given an array of integers, write a program to sort the array elements in: Ascending order (smallest to largest) Descending order (largest to smallest) The program should display the sorted array in both orders.
	Input: Array = [4, 2, 7, 1] Output: Ascending Order: [1, 2, 4, 7] Descending Order: [7, 4, 2, 1] Input: Array = [10, 5, 3, 8, 2] Output: Ascending Order: [2, 3, 5, 8, 10] Descending Order: [10, 8, 5, 3, 2]
24 (E)	Simple Calculator This program acts like a basic calculator, similar to the one on a mobile phone. It is designed in such a way that anyone—even a person who does not know coding—can understand how it works. The calculator: - Takes two numbers from the user - Asks the user what calculation to perform - Performs the calculation - Shows the final answer
	Input: First number: 8 Second number: 4 Operation: *

	Output: 32
25 (E)	Find Two Elements with Given Sum Given an array of numbers and a target value , find two numbers in the array whose sum equals the target . - Return the indices (positions) of these two numbers. - Indices start from 0. - Assume there is exactly one solution.
	Input: arr = [2, 4, 10, 6, 7] target = 6 Output: [0, 1] Input: arr = [3, 5, 1, 7] target = 8 Output: [2, 3]
26 (E)	Rearrange Array Alternately Given an array of even length , rearrange the elements such that: - Elements from the first half and second half are placed alternately. - The first element comes from the first half, the next from the second half, and so on.
	Input: [1, 2, 3, 4, 5, 6] Output: [1, 4, 2, 5, 3, 6] Input: [10, 20, 30, 40] Output: [10, 30, 20, 40]
27 (M)	Happy Number A Happy Number is a positive integer that eventually reaches 1 when repeatedly replaced by the sum of the squares of its digits . - If the process reaches 1, the number is Happy. - If the process enters a loop that does not include 1, the number is not Happy. Write a program to determine whether a given number is a Happy Number .
	Input: 19 Explanation: $19 \rightarrow 1^2 + 9^2 = 82$ $82 \rightarrow 8^2 + 2^2 = 68$ $68 \rightarrow 6^2 + 8^2 = 100$ $100 \rightarrow 1^2 + 0^2 + 0^2 = 1$ Output: Happy Number Input: n = 4 Explanation: $4 \rightarrow 4^2 = 16$ $16 \rightarrow 1^2 + 6^2 = 37$ $37 \rightarrow 3^2 + 7^2 = 58$ $58 \rightarrow 5^2 + 8^2 = 89$ $89 \rightarrow 8^2 + 9^2 = 145$ $145 \rightarrow 1^2 + 4^2 + 5^2 = 42$ $42 \rightarrow 4^2 + 2^2 = 20$ $20 \rightarrow 2^2 + 0^2 = 4$ Output: Not a Happy Number
28 (M)	Largest 5-Digit Prime Containing a Specific Digit Twice Program Definition

	Write a program to find the largest 5-digit prime number that contains a given digit exactly twice. Constraints: - The digit n is between 0 and 9. - Consider numbers in descending order from 99999 to 10000. - For each number: - Check if the number is prime. - Check if the number contains the digit n exactly two times. - Print the first number that satisfies both conditions.
	Input: n = 3 Output: 99833 Input: n = 7 Output: 99787
29 (M)	LEGO Tower (Alternate Color Stack) Program Definition: You are given two arrays red[] and blue[]. Each index represents one LEGO block position, where red[i] is the height of a red block and blue[i] is the height of a blue block at the same position. Your task is to find the maximum height of a tower that can be built by stacking blocks using the following rules: - You can start the tower with either red or blue block. - The colors of the blocks must alternate (red → blue → red → ... or blue → red → blue → ...). - From each index, only one block can be used (either red[i] or blue[i]), and each block can be used only once.
	Input: red = [3, 5, 2] blue = [4, 1, 6] Output: 14 Explanation - Start with blue (4), then red (5), then blue (6). - otal height = 4 + 5 + 6 = 14 Input: red = [2, 7, 4, 1] blue = [3, 5, 6, 2] Output: 18 Explanation - Start with red and alternate colors: - Red (2) - Blue (5) - Red (4) - Blue (2) - Total height = 2 + 5 + 4 + 2 = 18
30 (E)	Pivot Index Given an array of integers, write a program to find the pivot index. A pivot index is an index i in the array such that: - The sum of all elements to the left of i is equal to the sum of all elements to the right of i. - The element at the pivot index itself is not included in either sum. - If such an index exists, return the first pivot index found. - If no pivot index exists, print a message indicating that it was not found.
	Input: arr = [1, 7, 3, 6, 5, 6] Output: 3

	Explanation - Pivot index = 3 - Left sum = 1 + 7 + 3 = 11 - Right sum = 5 + 6 = 11 - Both sums are equal, and pivot element 6 is excluded. Input: arr = [1, 2, 3] Output: Pivot index not found Explanation - No index satisfies the left and right sum condition.
31 (E)	Electricity Bill Calculator Write a program to calculate the electricity bill based on the number of units consumed, using the following slab rates: First 30 units: ₹2.70 per unit Next 70 units (31–100): ₹4.00 per unit Next 100 units (101–200): ₹5.40 per unit Above 200 units: ₹6.40 per unit Additional Rule: - If the total calculated bill is less than ₹125, set the bill to a minimum charge of ₹125. The program should: - Take the number of units consumed as input. - Calculate the total bill according to the slab rates. - Apply the minimum bill rule. - Print the final amount to be paid.
	Input: Units consumed = 50 Output: Bill amount = ₹161.0 Input: Units consumed = 20 Output: Bill amount = ₹125.0
32 (E)	Print a Pyramid Star Pattern Write a program to print a pyramid pattern of stars (*) using a for loop.
	Input: 5 Output: <pre> * * * * * * * * * * * * * * *</pre>
33 (M)	Maximum Sum of Contiguous Subarray of Size k Program Definition Given an array of integers arr[] and an integer k, find the maximum sum of any contiguous subarray of size k. - A contiguous subarray is a subarray with consecutive elements. - The program should return the largest sum possible among all subarrays of length k.
	Input: arr = [2, 1, 5, 1, 3, 2] k = 3 Output: Maximum sum = 9 Explanation Subarray [5, 1, 3] has the highest sum of 9.

	Input: arr = [1, 2, 3, 4, 5, 6] k = 2 Output: Maximum sum = 11 Explanation Subarray [5, 6] has the highest sum of 11.
34 (M)	Longest Common Prefix Program Definition Given an array of strings, write a function to find the longest common prefix shared among all the strings. - A common prefix is a sequence of characters that appears at the beginning of every string in the array. - If no common prefix exists, return an empty string ("").
	Input: str = ["flowers", "flow", "fly", "flight"] Output: "fl" Input: str = ["dog", "cat", "animal", "monkey"] Output: ""
35 (E)	Count Duplicate Numbers in an Array A program was written to identify and count the number of duplicate elements present in a given array of integers. An element was considered a duplicate if it appeared more than once in the array. Each duplicate element was counted only once, regardless of how many times it appeared.
	Input: [1, 2, 3, 2, 4, 1, 5, 1] Output: 2
36 (E)	Minimum Training Hours to Equalize Skills You are given an array representing the skill levels of students. Each student has an integer skill value. - In one hour of training, a student's skill level can be increased by 1. - Skill levels cannot be decreased. - Training can be given to any student for any number of hours. Your task is to find the minimum total number of training hours required to make the skill level of any two students equal.
	Input: skills = [1, 3, 6] Output: 2 Input: skills = [4, 10, 7, 6] Output: 1 Input: skills = [5, 3, 5, 8] Output: 0
37 (H)	Given two sorted arrays of sizes m and n respectively, the task is to find the element that would be at the k-th position in the final sorted array formed by merging these two arrays.
	Input: a[] = [2, 3, 6, 7, 9], b[] = [1, 4, 8, 10], k = 5 Output: 6 Explanation: The final sorted array is [1, 2, 3, 4, 6, 7, 8, 9, 10] The 5th element is 6.
38 (E)	Given the number of rows and columns, print the corresponding swastika pattern using loops. Note: The number of rows and columns should be the same and an odd number. This will generate a perfect swastika pattern.



39 (M)	Write a program to print Pascal triangle .
	Input : N = 5 Output: <pre> 1 1 1 1 2 1 1 3 3 1 1 4 6 4 1 </pre>
40 (E)	Find the difference between the second largest element and the second smallest element of an array.
	Input: Enter the size of array: 7 Enter 7 elements: 5 1 9 7 1 5 3 Output: Difference: 4
41 (E)	Given the principal amount (P), rate of interest per annum (R), and loan tenure in months (N), write a program to calculate the EMI (Equated Monthly Installment) for a loan.
	Input : Enter principal amount (P): 500000 Enter annual interest rate (R): 7.5 Enter loan tenure in months (N): 60 Output: The EMI per month is: 10013.05
42 (E)	Given a number n then print n terms of fibonacci series in reverse order.
	Input : n = 5 Output : 3 2 1 1 0
43 (E)	Given an array arr[], the task is to reverse the array. Reversing an array means rearranging the elements such that the first element becomes the last, the second element becomes second last and so on.
	Input: arr[] = {1, 4, 3, 2, 6, 5} Output: {5, 6, 2, 3, 4, 1}
44 (E)	Given a string S and a character 'c', the task is to count the occurrence of the given character in the string.
	Input: S = "geeksforgeeks" and c = 'e' Output: 4 Explanation: 'e' appears four times in str.
45 (M)	Find if a given string can be represented from a substring by iterating the substring "n" times.
	Input: str = "abcabcabc" Output: true
46 (H)	Find the length of the maximum number of consecutive numbers jumbled up in an array.
	Input: arr[] = {1, 94, 93, 1000, 5, 92, 78}; Output : 3

	Explanation: The largest set of consecutive elements is 92, 93, 94.
47 (M)	Given an array arr[], the task is to find the subarray that has the maximum sum and return its sum.
	Input: arr[] = {2, 3, -8, 7, -1, 2, 3} Output: 11 Explanation: The subarray {7, -1, 2, 3} has the largest sum 11.
48 (M)	Write a program to merge two arrays into one, but if any element is repeated in the arrays, it should appear only once, and the duplicates should be replaced with zeros.
	Input: Enter the size of first array: 4 Enter the elements of first array: 7 2 5 9 Enter the size of second array: 5 Enter the elements of second array: 5 9 3 7 8 Output: Merged array: 7 2 5 9 0 3 8 0 0
49 (E)	Given a 2D square matrix, find the sum of elements in Principal and Secondary diagonals. For example, consider the following 4 X 4 input matrix.
	Input: 4 1 2 3 4 4 3 2 1 7 8 9 6 6 5 4 3 Output: Principal Diagonal: 16 Secondary Diagonal: 20
50 (E)	Given a string and write a C program to count the number of vowels and consonants in this string.
	Input: str = "geeks for geeks" Output: Vowels: 5 Consonants: 8
51 (E)	Given a string that contains both upper and lower case characters in it. The task is to count a number of upper- and lower-case characters in it
	Input: Introduction to Python Output: Lower Case characters : 18 Upper case characters : 2
52 (M)	Write a program to sort the numbers in a string expression where numbers are separated by the '+' sign. The program should rearrange the numbers in nondecreasing order while keeping them separated by '+'.
	Input: 1+1+3+1+3 Output: 1+1+1+3+3
53 (E)	Given two arrays a[] and b[], the task is to find the intersection of the two arrays. Intersection of two arrays is said to be elements that are common in both arrays. The intersection should not count duplicate elements and the result should contain items in any order
	Input: a[] = {1, 2, 1, 3, 1}, b[] = {3, 1, 3, 4, 1} Output: {1, 3} Explanation: 1 and 3 are the only common elements and we need to print only one occurrence of common elements.
54 (M)	Write a program to generate very big (at least 11 digits) and calculate the total of all those numbers. In the program, User will input the first digit i.e. "D", input count i.e. "N" which need to generate automatically sequential bases from 1 to N, other remaining digits should be 0 Input Validation $0 \leq D \leq 9$

	$1 < N \leq 1000$ Explanation Let's assume the inputs from the user for D is 5 and N is 50 • So the generated numbers will be 500000000001, 500000000002, 500000000003, 500000000004.... 500000000050. • The calculated sum of above generated numbers is 25000000001275. • So output of this program should be 25000000001275.
	Input: D: 3 N: 10 Output: 3000000000055
55 (M)	Given a time in the format of hh:mm (12-hour format) $0 < hh < 12$, $0 \leq mm < 60$. The task is to convert it into words as shown:
	Input: h = 5, m = 0 Output: five o' clock Input: h = 6, m = 24 Output: twenty-four minutes past six 5:00 → five o' clock 5:01 → one minute past five 5:10 → ten minutes past five 5:15 → quarter past five 5:30 → half past five 5:40 → twenty minutes to six 5:45 → quarter to six 5:47 → thirteen minutes to six 5:28 → twenty-eight minutes past five
56 (M)	Given a string that contains a special character together with alphabets ('a' to 'z' and 'A' to 'Z'), reverse the string in a way that special characters are not affected.
	Input: a!!!b.c.d,e'f,ghi Output: i!!!h.g.f,e'd,cba Input: str = "Ab,c,de!\$"
	Output: str = "ed,c,bA!\$"
57 (E)	Create a class that stores values using string keys and provides access to those values through a custom indexer. The indexer should: • Allow storing a value using a string key. • Allow retrieving a value using the same string key. • Return "Key Not Found" if the specified key does not exist.
	Input: store["Language"] = "C#"; store["Database"] = "SQL Server"; store["Language"]; store["Database"]; store["Python"]; Output: C# SQL Server Key Not Found
58 (M)	Write a program to accept a number between 1 and 9999 and display its equivalent in words. The program should convert the given number into its English word representation.
	Input: 512 Output: Five Hundred Twelve

59 (E)	Write a function to validate whether a password is strong. A password is considered strong if it satisfies all of the following conditions: • Minimum 8 characters long. • Contains at least one uppercase letter (A–Z). • Contains at least one digit (0–9). • Contains at least one special character (e.g., @, #, \$, %, &, *, !). Return "Valid Password" if all conditions are met; otherwise, return "Invalid Password".
	Input: Password = Abc@1234 Output: Valid Password Input: Password = Password = abcdef12 Output: Invalid Password
60 (H)	Write a program to find the maximum product of a contiguous subarray in a given integer array. The array may contain positive numbers, negative numbers, and zeros. The subarray must contain at least one element.
	Input: Array = [2, 3, -2, 4] Output: 6 Input: Array = [-2, 0, -1] Output: 0 Input: Array = [-2, 3, -4] Output: 24
61 (M)	Write a program to find the row index (0-based) that contains the maximum number of 1s in a row-wise sorted binary matrix. Each row of the matrix is sorted in non-decreasing order (all 0s appear before 1s). If two or more rows contain the same maximum number of 1s, return the smallest row index.
	Input: Matrix = [[0, 0, 1, 1], [0, 1, 1, 1], [0, 0, 0, 1]] Output: 1 Input: Matrix = [[0, 0], [1, 1]] Output: 1 Input: Matrix = [[1, 1, 1], [0, 0, 1], [0, 0, 0]] Output: 0
62 (H)	Write a program to calculate the total amount of rainwater that can be trapped between bars after raining. You are given an array of non-negative integers representing the height of bars in an elevation map, where the width of each bar is 1. Solve the problem using the two-pointer approach.
	Input: Height = [0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1] Output: 6 Input: Height = [4, 2, 0, 3, 2, 5] Output: 9 Input: Height = [3, 0, 3] Output: 3 Input: Height = [1, 0, 1] Output: 0

Output: 1

Input: Height = [0, 0, 0]

Output: 0

**63
(M)**

Write a program to print a **square pattern** of size n where:

- The **first and last rows** contain the row number repeated.
- The **first and last columns** contain the corresponding row number.
- The **interior** is filled concentrically with alphabets:
 - A for the outermost inner ring.
 - B for the next inner ring.
 - C for the next inner ring, and so on.

Input: n = 7

Sample output (n = 7):

1	1	1	1	1	1	1
2	A	A	A	A	A	2
3	A	B	B	B	A	3
4	A	B	C	B	A	4
5	A	B	B	B	A	5
6	A	A	A	A	A	6
7	7	7	7	7	7	7

**64
(E)**

Write a program to move all **0s** to the **end** of a given integer array while maintaining the **relative order of non-zero elements**.

The operation must be performed **in-place** without creating a copy of the array.

Input: Array = [0, 1, 0, 3, 12]

Output: [1, 3, 12, 0, 0]

Input: Array = [0, 0, 0, 1]

Output: [1, 0, 0, 0]

Input: Array = [1, 2, 3]

Output: [1, 2, 3]

Input: Array = [0]

Output: [0]

**65
(M)**

Write a program to find the **length of the longest substring** in a given string that contains **no repeating characters**.

A **substring** is a contiguous sequence of characters within the string.

Input: String = "abcabcbb"

Output: 3

Input: String = "bbbbbb"

Output: 1

Input: String = "pwwkew"

Output: 3

**66
(E)**

Write a function to count the number of **set bits (1s)** in the binary representation of a given **unsigned integer**.

Solve the problem using **Brian Kernighan's Algorithm**.

Input: n = 11

Output: 3

	Explanation: The binary representation of 11 is 1011, which contains 3 set bits. Input: n = 128 Output: 1 Explanation: The binary representation of 128 is 10000000, which contains 1 set bit. Input: n = 255 Output: 8 Explanation: The binary representation of 255 is 11111111, which contains 8 set bits.
67 (H)	String to Integer (myAtoi) GNWebsoft Write a program to implement the myAtoi(string s) function, which converts a given string into a 32-bit signed integer. Do not use the built-in atoi() function. The conversion should follow these rules: Ignore leading whitespace. Determine the sign: - If the next character is '-', the number is negative. - If the next character is '+', the number is positive. - Otherwise, assume the number is positive. Read the digits: - Skip leading zeros. - Continue reading digits until a non-digit character or the end of the string is reached. - If no digits are found, return 0. - If the converted value is outside the 32-bit signed integer range: - Return -2147483648 if it is less than INT_MIN. - Return 2147483647 if it is greater than INT_MAX.
	Input: String = " -42" Output: -42 Input: String = "42" Output: 42 Input: String = " +00123abc" Output: 123 Input: String = "words and 987" Output: 0
68 (H)	Write a program to find the total number of contiguous subarrays in a given integer array whose sum is equal to k. A subarray is a contiguous sequence of elements within an array.
	Input: Array = [1, 1, 1] k = 2 Output: 2 Input: Array = [1, 2, 3] k = 3 Output: 2 Input: Array = [1, -1, 0]

	k = 0 Output: 3 Input: Array = [3, 4, 7, 2, -3, 1, 4, 2] k = 7 Output: 4
69 (E)	Write a program to find the minimum number of moves required to make any two elements in a given integer array equal. In one move, you can increment or decrement an element by 1. Return the minimum number of moves required.
	Input: Array = [1, 3, 5, 2] Output: 1 Input: Array = [10, 20, 30] Output: 10 Input: Array = [5, 5, 8] Output: 0

E (24)	2, 3, 9, 17, 19, 22, 24, 25, 26, 30, 31, 32, 35, 36, 38, 40, 41, 42, 43, 44, 49, 50, 51, 53, 57, 59, 64, 66, 69
M (33)	6, 8, 11, 13, 14, 37, 46, 60, 62, 67, 68
H (12)	1, 4, 5, 7, 10, 12, 15, 16, 18, 20, 21, 23, 27, 28, 29, 33, 34, 39, 45, 47, 48, 52, 54, 55, 56, 58, 61, 63, 65